Distributed Training Experiment Manual – Iluvatar V150

1. Objective

This experiment trains the Qwen3-06b model using the FlagOS components **FlagScale**, **FlagCX**, **TransformerEngine-FL**, and **FlagGems**. The FlagOS components are already installed at:

```
代码块
1  /nfs/tianxiao/iluvatar/
```

2. Experiment Steps

1. Log in to the online laboratory and enter the IDE:

```
代码块
1  https://docs.flagos.io/projects/onlinelaboratory/zh-
   cn/latest/onlinelaboratory.html
```

2. Enter the FlagScale directory:

```
代码块
1  cd /nfs/tianxiao/iluvatar/FlagScale/build/BI_V150/FlagScale
```

3. FlagScale uses Hydra for configuration management. Configurations are divided into two levels: outer experiment-level YAML files and inner task-level YAML files.
   - **Experiment-level YAML file:** defines the experiment directory, backend engine, task type, and other environment-related configurations. In this experiment, the configured task type is **training** and the config file is `train.yaml`. To modify the config:

```
代码块
1  vim examples/qwen3/conf/train.yaml
```

代码块

```yaml
exp_name: Qwen3-06b-Train
seed: 42
save_steps: 10000
load: null
exp_dir: ./exps/${experiment.exp_name}
ckpt_format: torch
task:
  type: train
  backend: megatron
  entrypoint: flagscale/train/train_gpt.py
  #entrypoint: third_party/Megatron-LM/pretrain_gpt.py
runner:
  per_node_task: false
  no_shared_fs: false
  rdzv_backend: static
  hostfile: null
  ssh_port: 7788
  nproc_per_node: 2 # Set to 2 cards; the lab instance is a 2-GPU environment
cmds:
  before_start: source  /public-nvme/tianxiao/06B/iluvatar/FlagScale/build/BI_V150/FlagScale/examples/qwen3/conf/env.sh
  #before_start: ulimit -n 1048576 && source /root/miniconda3/bin/activate flagscale-train
envs:
  LOGLEVEL: "INFO"
  CUDA_VISIBLE_DEVICES: "0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15"
  CUDA_DEVICE_MAX_CONNECTIONS: 1

action: run

hydra:
  run:
    dir: ${experiment.exp_dir}/hydra
```

- **Task-level YAML file:** specifies the model, dataset, and parameters for specific tasks such as training or inference. In this experiment, you configure the training-related task parameters — for example, the number of training steps is computed via `global_batch_size` and `train_samples`. To modify the config, use the vim command:

$$\text{Total Steps} = \frac{\text{train_samples}}{\text{global_batch_size}} = \frac{640}{32} = 20\text{步}$$

Training step calculation: Total Steps = train_samples / global_batch_size, e.g., 640 / 32 = 20 steps.

代码块

```
vim examples/qwen3/conf/train/0_6b_te_fl.yaml
```

- Enter edit mode: press `i`
- Save and exit: press `Esc` → type `:wq` → press `Enter`

代码块

```
system:
  no_shared_fs: ${experiment.runner.no_shared_fs}
  num_workers: 0
  tensor_model_parallel_size: 1
  pipeline_model_parallel_size: 1
  context_parallel_size: 1
  disable_bias_linear: true
  reset_position_ids: True
  reset_attention_mask: True
  qk_layernorm: true # true
  sequence_parallel: true
  use_distributed_optimizer: true
  overlap_grad_reduce: true
  overlap_param_gather: true
  finetune: false
  precision:
    bf16: true
    attention_softmax_in_fp32: true
    accumulate_allreduce_grads_in_fp32: true
  logging:
    log_interval: 1
    tensorboard_log_interval: 1
    wandb_project: ${experiment.exp_name}
    wandb_exp_name: ${experiment.exp_name}
    log_timers_to_tensorboard: true
    log_validation_ppl_to_tensorboard: true
    log_throughput: true
    log_params_norm: true
    log_num_zeros_in_grad: true
    log_memory_to_tensorboard: true
  checkpoint:
```

```yaml
    save_interval: ${experiment.save_steps}
    load: ${experiment.load}
    ckpt_format: ${experiment.ckpt_format}

model:
  profile: true
  use_pytorch_profiler: true
  profile_step_start: 3
  profile_step_end: 4
  profile_ranks: [0,5]
  tensorboard_dir: /public-nvme/tianxiao/06B/iluvatar/
  transformer_impl: transformer_engine
  #### FlagOS-Related Arguments for unified computation backend and communication backend
  te_fl_prefer: flagos #flagos # enable flagos:triton in transformer engine fl
  #te_fl_per_op: get_flash_attention_class=reference #;get_attention_backend=reference;multi_tensor_adam=reference;generic_gemm=reference;rmsnorm_bwd=reference;rmsnorm_fwd=reference #reference # enable custom ops selection for transformer engine fl
  #te_fl_allow_vendors: None #"vendor_a" # allow vendors for transformer engine fl
  #te_fl_deny_vendors: None #"vendor_b" # deny vendors for transformer engine fl
  enable_flag_gems: true # enable flag gems to replace torch ops for distributed training
  flag_gems_log_path: /tmp/gems.txt # path of flag gems logging
  flag_gems_unused: [] #[] # flag gems unused ops list
  distributed_backend: flagcx #flagcx/nccl # enable flagcx for distributed training
  #### FlagOS-Related Arguments
  num_layers: 28
  hidden_size: 1024
  ffn_hidden_size: 3072
  kv_channels: 128
  num_attention_heads: 16
  group_query_attention: true #true
  num_query_groups: 8 # num_key_value_heads
  seq_length: 4096 #4096
  max_position_embeddings: 40960
  norm_epsilon: 1e-6
  use_rotary_position_embeddings: true
  rotary_base: 1000000
  swiglu: true
  normalization: RMSNorm
  init_method_std: 0.006 #0.02
  attention_dropout: 0.0
```

```
    hidden_dropout: 0.0
    clip_grad: 1.0
    position_embedding_type: rope
    untie_embeddings_and_output_weights: false
    no_position_embedding: true
    no_rope_fusion: true

    # training
    seed: ${experiment.seed}
    # finetune: false
    micro_batch_size: 1 #4
    global_batch_size: 32 #2048
    eval_iters: 0
    # train_iters: 20
    train_samples: 3200 #244141056 #1T #29297664 #120B tokens # since global_batch_size is 32, 3200 equals 100 steps

    optimizer:
      weight_decay: 0.1
      adam_beta1: 0.9
      adam_beta2: 0.95
      lr_scheduler:
        lr: 3.0e-3
        min_lr: 3.0e-4
        lr_warmup_samples: 64 #2048000
        lr_decay_style: cosine


data:
    data_path: /nfs/tianxiao/iluvatar/datas/Nemotron-CC-v2__High-Quality__part1_text_document
    split: 1
    no_mmap_bin_files: true
    tokenizer:
      legacy_tokenizer: true #true
      tokenizer_type: QwenTokenizerFS
      tokenizer_path: /nfs/tianxiao/iluvatar/datas/qwentokenizer
      vocab_size: 151851
      make_vocab_size_divisible_by: 64
```

4. Start the training.

代码块

```
python3 run.py \
    --config-path /conf \
```

```
    --config-name train \
    action=run \
    experiment.exp_dir=/conf/Qwen3-06b-Train \
    experiment.save_steps=20
```

Parameter	Description
python3 run.py	The main command to start training; it runs the run.py script with the Python 3 inte
--config-path /conf	Specifies the directory of the configuration files; the script looks for configurations
--config-name train	Specifies the name of the main configuration file to use, i.e., loads /conf/train.yaml configuration.
action=run	Specifies the action to execute; run means start training.
experiment.exp_dir=...	Sets the root directory of the experiment; all outputs such as logs and model check here.
experiment.save_steps=20	Sets the model saving frequency, i.e., saves a checkpoint every 20 training steps.

5. Open another window and monitor the checkpoints.

代码块

```
watch -n 2 ls -lth /conf/Qwen3-06b-Train/checkpoints/
```

代码块

```
Every 2.0s: ls -lth /conf/Qwen3-06b-Train/checkpoints/

                    13d087e951ad: Wed Jul 22 13:46:34 2026

total 8.0K
-rw-r--r-- 1 root root 16 Jul 22 13:44 latest_wandb_artifact_path.txt
-rw-r--r-- 1 root root  3 Jul 22 13:44 latest_checkpointed_iteration.txt
drwxr-xr-x 3 root root 24 Jul 22 13:44 iter_0000100
drwxr-xr-x 3 root root 32 Jul 22 13:34 iter_0000080
drwxr-xr-x 3 root root 24 Jul 22 13:24 iter_0000060
drwxr-xr-x 3 root root 32 Jul 22 13:14 iter_0000040
drwxr-xr-x 3 root root 24 Jul 22 13:05 iter_0000020
```

- `iter_XXXXX/` : each directory corresponds to a checkpoint, containing the model weights, optimizer state, etc.
- `latest_checkpointed_iteration.txt` : records the latest saved iteration step (100).

- `latest_wandb_artifact_path.txt`: records the path of the latest artifact synced to WandB (if WandB is enabled).

6. View the results — the top-level structure of the checkpoint.
   - View the model weights: enter the Python environment with `python3` and inspect the top-level structure of the checkpoint:

```
代码块
1  import sys
2
3  # Add both paths
4  sys.path.append("/nfs/tianxiao/iluvatar/FlagScale/build/BI_V150/FlagScale
   ")
5  sys.path.append("/nfs/tianxiao/iluvatar/FlagScale/build/BI_V150/FlagScale
   /third_party/Megatron-LM")
6
7  # Verify that the megatron module can be found
8  import importlib.util
9  spec = importlib.util.find_spec("megatron")
10 print("megatron module found at:", spec.origin if spec else "NOT FOUND")
11
12 # If found, load the checkpoint
13 import torch
14 ckpt = torch.load(
15     "/conf/Qwen3-06b-
   Train/checkpoints/iter_0000040/mp_rank_00/model_optim_rng.pt",
16     map_location="cpu",
17     weights_only=False
18 )
19
20 print("Top-level keys:", ckpt.keys())
```

```
代码块
1  Top-level keys: dict_keys(['args', 'checkpoint_version', 'iteration',
   'model', 'optimizer', 'opt_param_scheduler', 'rng_state',
   'num_floating_point_operations_so_far'])
```

   - View the model weight details: continue with the following code to see which weights are stored in `model`:

```
代码块
1  # View the current iteration step
```

```
print("Iteration:", ckpt["iteration"])

# View the structure of the model weights (first 20)
print("\nModel keys (first 20):")
for i, (k, v) in enumerate(ckpt["model"].items()):
    if i >= 20:
        print("  ...")
        break
    if hasattr(v, "shape"):
        print(f"  {k}: {v.shape} | dtype={v.dtype}")
    else:
        print(f"  {k}: {type(v)}")

# Count the total number of parameters
total_params = sum(v.numel() for v in ckpt["model"].values() if
hasattr(v, "numel"))
print(f"\nTotal parameters: {total_params / 1e6:.2f}M")
```

- View the training parameters

代码块

```
# View the parameter configuration used during training
print("\nTraining args:")
print(ckpt["args"])
```

3. Open-ended Homework Experiment

Question: If I want to continue using AI for training optimization, which files should I feed to the AI for analysis?

- **The `args` config in the checkpoint:** print it with the "View the training parameters" command above; it contains all the hyperparameters (learning rate, batch size, model architecture, etc.). This is the foundation for AI analysis.
- **Training logs (logs):** contain the loss, learning rate, gradient norm, etc. for each iteration. The AI needs the trend of the loss curve to determine whether the training is proceeding normally.

代码块

```
# View the lines in the log that contain loss and iteration
grep -i "loss\|iteration" /conf/Qwen3-06b-
Train/logs/host_0_localhost.output | head -50
```

- **WandB / TensorBoard data:** if you enabled WandB (your config has `wandb_project='Qwen3-06b-Train'`), you can directly export the WandB metrics data (CSV or JSON format), and the AI can analyze the training dynamics from it.

（注：部分内容由豆包工作 AI 生成）